

CS P35- DATA STRUCTURES

UNIT IV

Graph: Terminology, Representation, Traversals - Applications - spanning trees, shortest path and Transitive closure, Topological sort. Sets: Representation - Operations on sets - Applications.

TWO MARKS

1. What is spanning tree?(Nov 2015)

A tree is a spanning tree of a connected graph $G(V,E)$ such that

- The edges in T form a tree which includes all the vertices of G
- Any tree consisting solely of edges in G and including all vertices in G is called spanning tree.

2. Write the advantage of topological sort? (Nov 2015).

A directed graph G in which the vertices represent tasks or activities and the edges represent activities to move from one event to another then the task is known as activity on vertex network or AOV-network/pert network

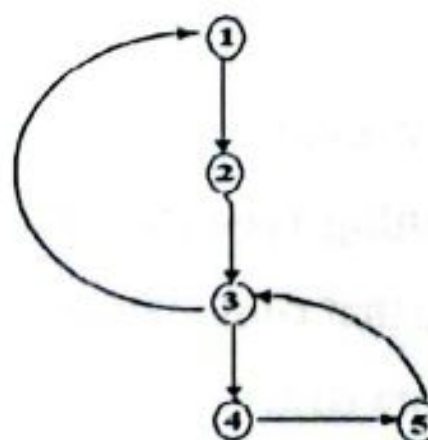
3. What are the two ways to maintain the graph G in the memory of the computer?(Dec 2014)

We can represent the graph in the memory of the computer using:

- (a).Adjacency Graph
- (b).Adjancency Matrix.

4. When the graph is said to be strongly connected?(Dec 2014)

A graph is said to be a strongly connected graph, if for every pair of distinct vertices there is a directed path from every vertex to every other vertex. It is also referred as a complete graph.



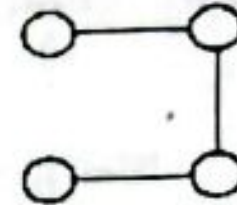
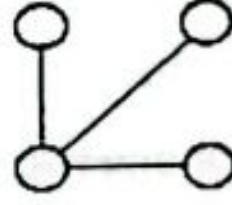
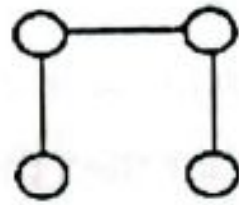
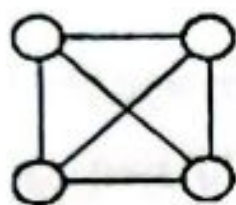
5. How does a graph differ from a Tree(Dec 2016)

Graph	Tree
1.It is a collection of Vertices and edges	1. It is a collection of nodes and edges.
2. A cycle can be formed.	2. There will not be any cycle.
3. Each node can have any number of edges.	3. General trees consist of the nodes having any number of child nodes.

6. Define Multistage Graph (Dec 2016)

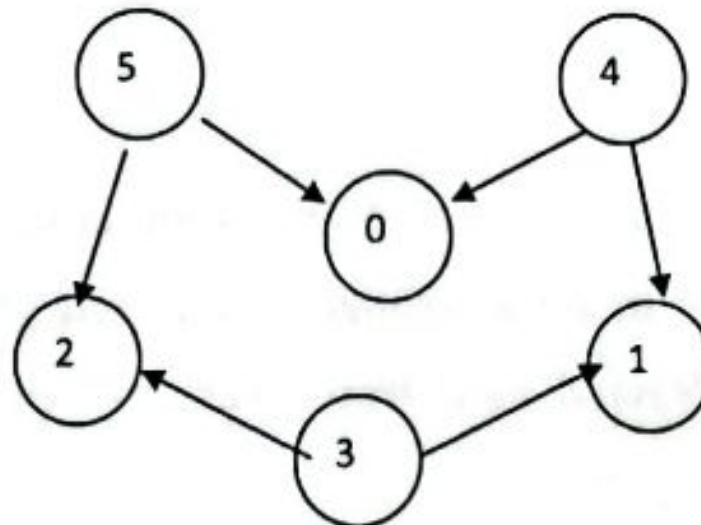
A multistage graph is a directed graph in which the nodes can be divided into a set stages such that all edges are from a stage to next stage only.

5. Draw a complete graph with 4 Vertices (Nov 2017)



6.In what situation topological ordering is not possible in a graph .justify?(Nov 2017)

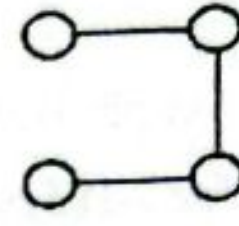
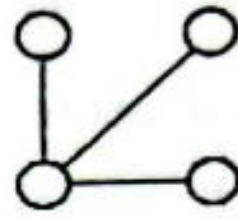
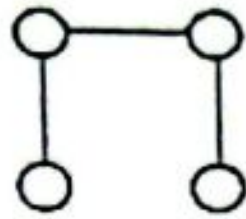
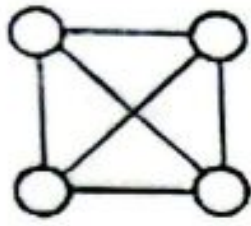
Topological sorting for a graph is not possible if the graph is not a DAG(Directly aconnected graph).



In the above graph the vertices are not connected properly.

7. Define Minimum Spanning Tree.(Nov 2018).

A spanning tree is any tree that consists only of edges in a graph G and that includes all the vertices in G at minimum cost.



8. List the properties of set equality relation (Nov 2018)

9. Write the advantage of topological sorting (Nov 2019).

The topological ordering can be used to quickly compute shortest paths through a weighted directed acyclic graph and it can also be used to show the precedence among events.

10. Define an elementary path in a graph. (Sep 2020)

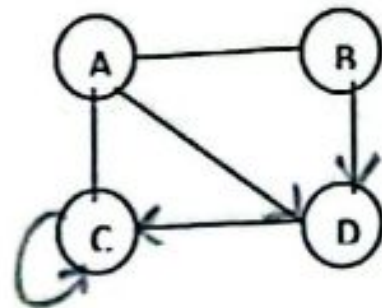
A path is called elementary if no vertices appear more than once in it....if the initial and the terminal vertices of a path are the same, that is $x_0 = x_n$, then the path is called a cycle.

11. Draw the graph with nodes and edges having the following adjacency matrix.

```

0 1 1 0
1 0 0 1
1 0 1 0
1 0 1 0

```



11 marks

1. Define a Graph. How it is represented? Explain its traversal scheme with an example. (Nov 2015, Nov 2017)

1. DEFINING GRAPH

- A graph G consists of a set V of vertices (nodes) and a set E of edges (arcs).
- We write $G=(V,E)$. V is a finite and non-empty set of vertices.
- E is a set of pair of vertices; these pairs are called as edges.
- Therefore $V(G)$, read as V of G , is a set of vertices and $E(G)$, read as E of G is a set of edges.
- An edge $e=(v,w)$ is a pair of vertices v and w , and to be incident with v and w .
- A graph can be pictorially represented as follows,



We have numbered the graph as 1,2,3,4.

Therefore, $V(G)=\{1,2,3,4\}$ and $E(G) = \{(1,2), (1,3), (2,3), (2,4), (3,4)\}$.

Given an undirected graph $G = (V,E)$ and a vertex v in $V(G)$ we are interested in visiting all vertices in G that are reachable from v (that is all vertices connected to v). We have two ways to do the traversal. They are

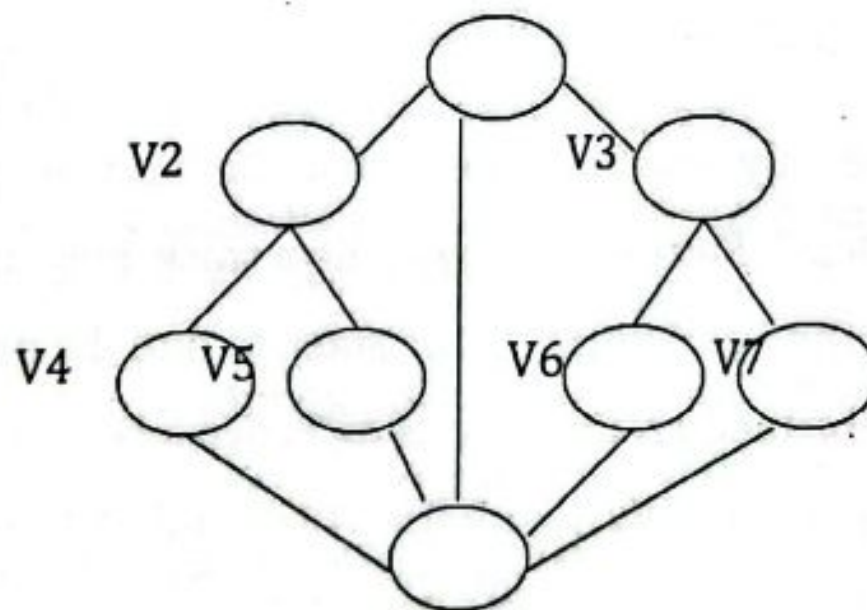
DEPTH FIRST SEARCH

In graphs, we do not have any start vertex or any special vertex singled out to start traversal from. Therefore the traversal may start from any arbitrary vertex.

We start with say, vertex v . An adjacent vertex is selected and a Depth First search is initiated from it, i.e. let $V_1, V_2, \dots, V_k$ are adjacent vertices to vertex v . We may select any vertex from this list. Say, we select V_1 . Now all the adjacent vertices to v_1 are identified and all of those are visited; next V_2 is selected and all its adjacent vertices visited and so on.

This process continues till all the vertices are visited. It is very much possible that we reach a traversed vertex second time. Therefore we have to set a flag somewhere to check if the vertex is already visited.

Let us see an example, consider the following graph.



Let us start with V_1 ,

1. Its adjacent vertices are V_2, V_8 , and V_3 . Let us pick on v_2 .
2. Its adjacent vertices are V_1, V_4, V_5 , V_1 is already visited.

3. Let us pick on V4.
4. Its adjacent vertices are V2, V8.
5. V2 is already visited .let us visit V8.
6. Its adjacent vertices are V4, V5, V1, V6, V7.
7. V4 and V1 are visited. Let us traverse V5.
8. Its adjacent vertices are V2, V8. Both are already visited therefore, we back track.
9. We had V6 and V7 unvisited in the list of V8. We may visit any. We may visit any. We visit V6.
10. Its adjacent is V8 and V3. Obviously the choice is V3.
11. Its adjacent vertices are V1, V7. We visit V7.
12. All the adjacent vertices of V7 are already visited, we back track and find that we have visited all the vertices.

Therefore the sequence of traversal is V1, V2, V4, V5, V6, V3, V7.

This is not a unique or the only sequence possible using this traversal method. We may implement the Depth First search by using a stack, pushing all unvisited vertices to the one just visited and popping the stack to find the next vertex to visit.

- This procedure is best described recursively as in, Procedure DFS(v)
// Given an undirected graph $G = (V, E)$ with n vertices and an array visited (n) initially set to zero . This algorithm visits all vertices reachable from v . G and VISITED are global
> //VISITED (v) = 1
for each vertex w
adjacent to v do if
VISITED (w) = 0 then call

DFS (w) end

end DFS

COMPUTING TIME

1. In case G is represented by adjacency lists then the vertices w adjacent to v can be determined by following a chain of links. Since the algorithm DFS would examine each node in the adjacency lists at most once and there are $2e$ list nodes. The time to complete the search is $O(e)$.
2. If G is represented by its adjacency matrix, then the time to determine all vertices adjacent to v is $O(n)$.

Since at most n vertices are visited. The total time is $O(n^2)$.

BREADTH FIRST SEARCH

- In DFS we pick on one of the adjacent vertices; visit all of its adjacent vertices and back track to visit the unvisited adjacent vertices.
- In BFS, we first visit all the adjacent vertices of the start vertex and then visit all the unvisited vertices adjacent to these and so on.
- Let us consider the same example, given in figure. We start say, with V_1 . Its adjacent vertices are V_2, V_8, V_3 .
- We visit all one by one. We pick on one of these, say V_2 . The unvisited adjacent vertices to V_2 are V_4, V_5 . we visit both.
- We go back to the remaining visited vertices of V_1 and pick on one of this, say V_3 . The unvisited adjacent vertices to V_3 are V_6, V_7 . There are no more unvisited adjacent vertices of V_8, V_4, V_5, V_6 and V_7 .

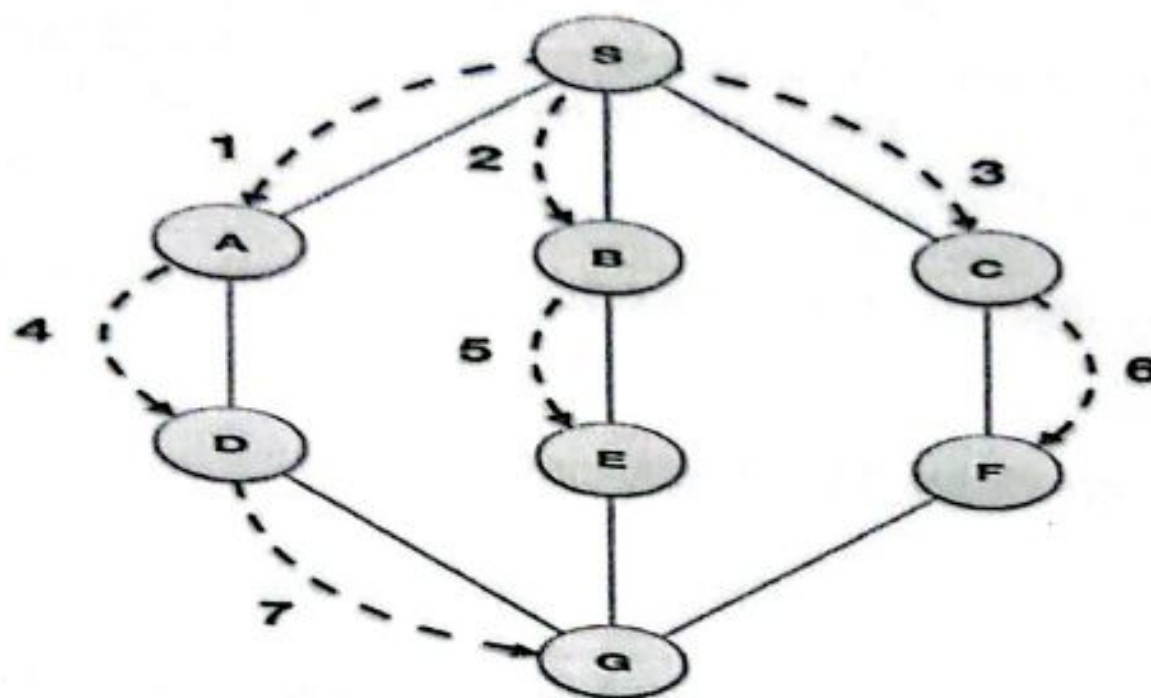


FIG: Breadth First Search

- Thus the sequence so generated is V1, V2, V3, V4, V5, V6, V7. Here we need a queue instead of a stack to implement it. (S A B C D E F G) correspondingly.

- We add unvisited vertices adjacent to the one just visited at the rear and read at front to find the next vertex to visit.

3. Give notes on (a).Representation of sets. (NOV 2015, APRIL 2018)

One of the most basic mathematical constructs is a set, which is an unordered collection of unique objects. The objects contained within a set are referred to as the set's elements.

Examples of this notation can be seen below: $S = \{ 1, 3, 5, 7, 9 \}$

$T = \{ \text{Scott, Jisun, Sam} \}$

$U = \{ -4, 3.14159, \text{Todd}, x \}$

The Fundamentals of Sets

The number of unique elements in a set is the set's *cardinality*. The set $\{1, 2, 3\}$ has cardinality 3, just as does the set $\{1, 1, 1, 1, 1, 1, 1, 2, 3\}$ (because it only has three unique elements). A set may have no elements in it at all. Such a set is called the *empty set*, and is denoted as $\{\}$ or $\emptyset$, and has a cardinality of 0.

A set, on the other hand, is *unordered* and contains *unique* items. Since sets are unordered, the elements of a set may be listed in any order. That is, the sets $\{1, 2, 3\}$ and $\{3, 1, 2\}$ are considered equivalent. Also, any duplicates in a set are considered redundant. The set $\{1, 1, 1, 2, 3\}$ and the set $\{1, 2, 3\}$ are equivalent. Two sets are equivalent if they have the same elements. (Equivalence is denoted with the $=$ sign; if S and T are equivalent they are written as $S = T$.)

(b).Operation on sets.(nov 2015, April 2018)

Set Operations

Common operations on numbers include addition, multiplication, subtraction, exponentiation, and so on. For sets, there are four basic operations:

Union – the union of two sets, denoted $S \cup T$, is akin to addition for numbers. The union operator returns a set that contains all of the elements in S and all of the elements in T . For example, $\{1, 2, 3\} \cup \{2, 4, 6\}$ equals $\{1, 2, 3, 2, 4, 6\}$. (The duplicate 2 can be removed to provide a more concise answer, yielding $\{1, 2, 3, 4, 6\}$.) Formally, $S \cup T = \{x : x \in S \text{ or } x \in T\}$. In English, this translates to S union T results in the set that contains an element x if x is in S or in T .

Intersection – the intersection of two sets, denoted $S \cap T$, is the set of elements that S and T have in common. For example, $\{1, 2, 3\} \cap \{2, 4, 6\}$ equals $\{2\}$, since that's the only element both $\{1, 2, 3\}$ and $\{2, 4, 6\}$ share in common. Formally, $S \cap T = \{x : x \in S \text{ and } x \in T\}$. In English, this translates to S intersect T results in the set that contains an element x if x is both in S and in T .

Difference – the difference of two sets, denoted $S - T$, are all of the elements in S that are not in T . For example, $\{1, 2, 3\} - \{2, 4, 6\}$ equals $\{1, 3\}$, since 1 and 3 are the elements in S that are not in T . Formally, $S - T = \{x : x \in S \text{ and } x \notin T\}$. In English, this translates to S set difference T results in the set that contains an element x if x is in S and not in T .

Complement – Earlier we discussed how typically sets are limited to a known universe of possible values, such as the integers. The complement of a set, denoted S' , is $U - S$. (Recall that U is the universe set.) If our universe is the integers 1 through 10, and $S = \{1, 4, 9, 10\}$, then $S' = \{2, 3, 5, 6, 7, 8\}$. (Complementing a set is akin to negating a number. Just like negating a number twice will give you the original number back—that is, $--x = x$ —complementing a set twice will give you the original set back— $S'' = S$.)

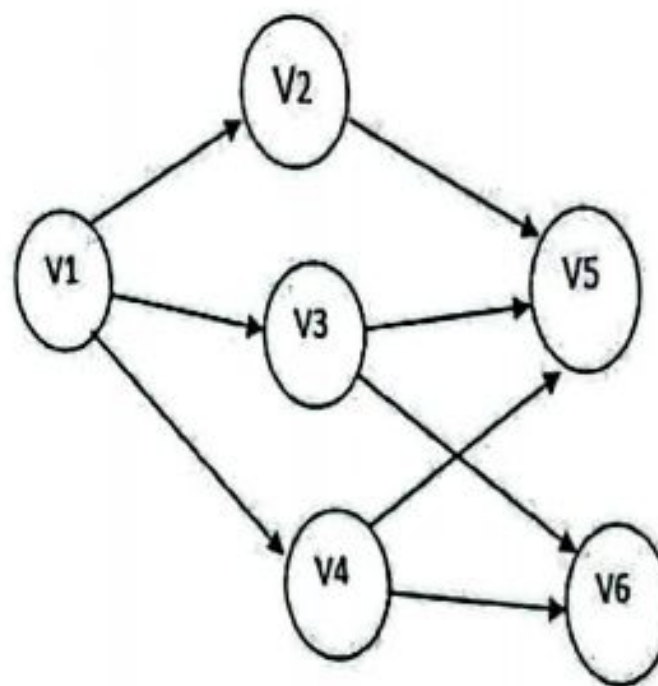
3. Explain the procedure to find the shortest path for the given graph. Develop and analyze your procedure (Dec 2014)

4. Describe the topological sort with an example (Dec 2014, April 2017, April 2018).

A directed graph G in which the vertices represent event and the edges represent activities to move from one event to another then that graph is known as activity on vertex network (or) AOV network (or) PERT graph.

> PERT graph is used to analyze interrelated activities of complete projects

- The purpose of topological sort is to order the events in a linear manner with the restriction that an event cannot precede other event that must first take place
- Let us assume that we have to do a set of jobs. But certain jobs have to be performed after completion of other job.
- Topological sort defines an order to do these jobs such that each job is performed only after all of its constraints are satisfied.
- Topological sorting is an ordering of the vertices in a directed acyclic graph, such that , if there is a path from x to y, then y appears after x and not x after y in a cycle.



ALGORITHM FOR TOPOLOGICAL SORTING

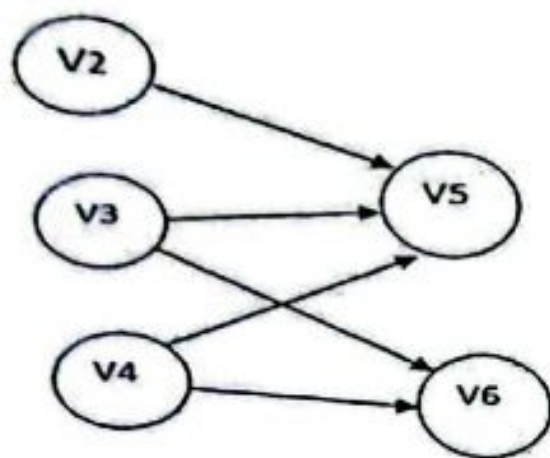
- Step 1 : For $l \rightarrow 1$ to n do // o/p the vertices // .
- Step 2 : If every vertex has no predecessor then (the net work has a cycle and stop).
- Step 3 : Pick a vertex which has no predecessor.
- Step 4 : Output V.
- Step 5 : Delete V and all edges leading and of V from the network.

➤ Step 6 : End.

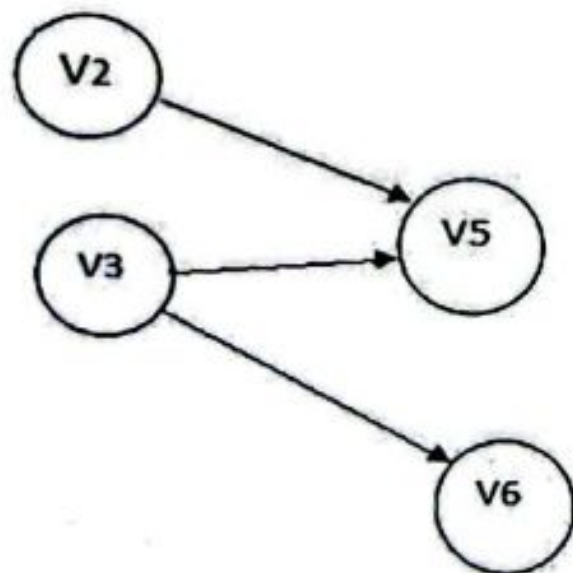
Applying this for above graph, we get:

▣ A) Initial

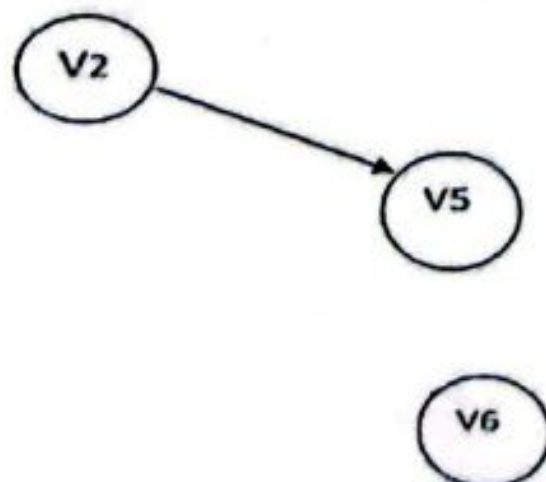
▣ B) V1



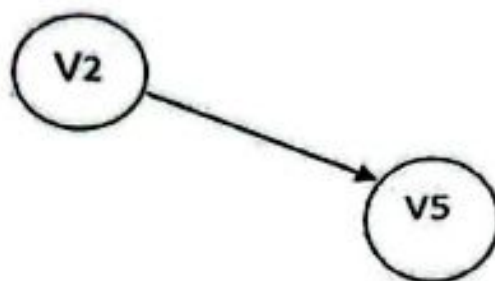
▣ C) V4



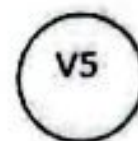
▣ D) V3



▣ E) V6



▣ F) V2



- G) V5
- Topological order generated is : V1, V4, V3, V6, V2, V5.
- In step B) we have 3 vertices V2, V3, & V4 which has no predecessors and any of these can be the next vertex in the topological order.
- In this problem, the functions are :
 - Decide whether a vertex has any predecessors.
 - Decide a vertex together with all its incident edges.
 - This can be efficiently done if for each vertex a count of the number of its immediate predecessor is kept.
 - This is easily implemented if the network is represented as a adjacency list.

- The following algorithm assumes that the network is represented as adjacency list.

ALGORITHM

Procedure TOPOLOGICAL_ORDER (count, vertex, link, n)

//The n vertices of an AOV network are listed in topological order. The network is represented as a set of adjacency lists with

1. COUNT(I) = the in-degree of vertex I //
2. Top <- 0 //initialize stack//
3. For I <- 1 to n do //create a linked stack of vertices with no predecessors//
4. If COUNT (I) = 0 then [COUNT(I) <- top; top <- I]
5. End

- 13.If COUNT (K) =0 //add vertex K to stack//
- 14.Then [COUNT (K) <- top; top <- K]
- 15.Ptr <- LINK (ptr)
- 16.End
- 17.End
18. End TOPOLOGICAL_ORDER.

6. Construct the minimum spanning for the following graph using Prim's and Kruskal Algorithm.(April 2017)

Start from an arbitrary vertex (root). At each stage, add a new branch (edge) to the tree already constructed; the algorithm halts when all the vertices in the graph have been reached.

Algorithm prims(e, cost, n, t)

{

Let (k, l) be an edge of
minimum cost in E; Mincost
:= cost[k, l];

T[1, 1] := k;

t[1, 2] := l;

For l := 1 to
n do

If (cost[i, l] < cost[i, k]) then

near[i] := l; Else

near[i] := k;

Near[k] := near[l] := 0;

For i := 2 to n-1 do

{

Let j be an index such that near[j] ≠ 0 and

Cost[j, near[j]] is

minimum; T[i, 1] := j;

t[i, 2] := near[j];

Mincost := mincost +

Cost[j, near[j]];

Near[j] := 0;

For $k:=0$ to n do

If $\text{near}((\text{near}[k] \neq 0) \text{ and } (\text{Cost}[k, \text{near}[k]] > \text{cost}[k, j]))$ then

Near[k]:=j;

}

Return mincost;

}

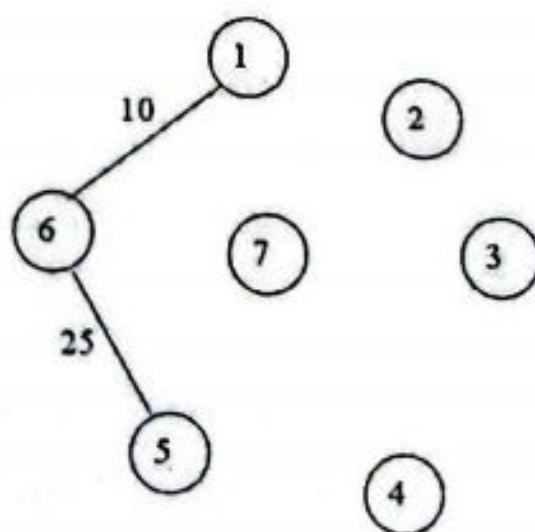
The prims algorithm will start with a tree that includes only a minimum cost edge of G .

Then, edges are added to the tree one by one. the next edge (i, j) to be added in such that i is a vertex

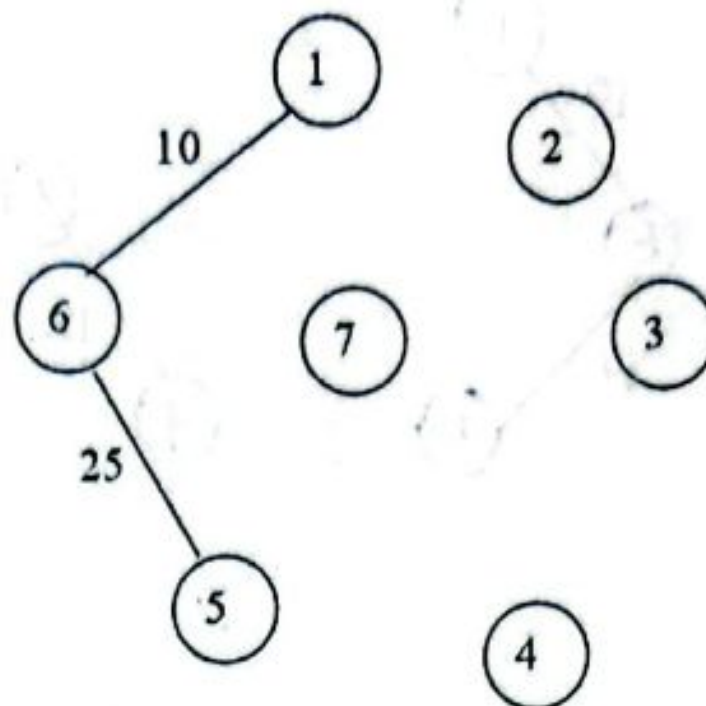
included in the tree, j is a vertex not yet included, and cost of (i, j) , $\text{cost}[i, j]$ is minimum among all the edges.

The working of prims will be explained by following diagram

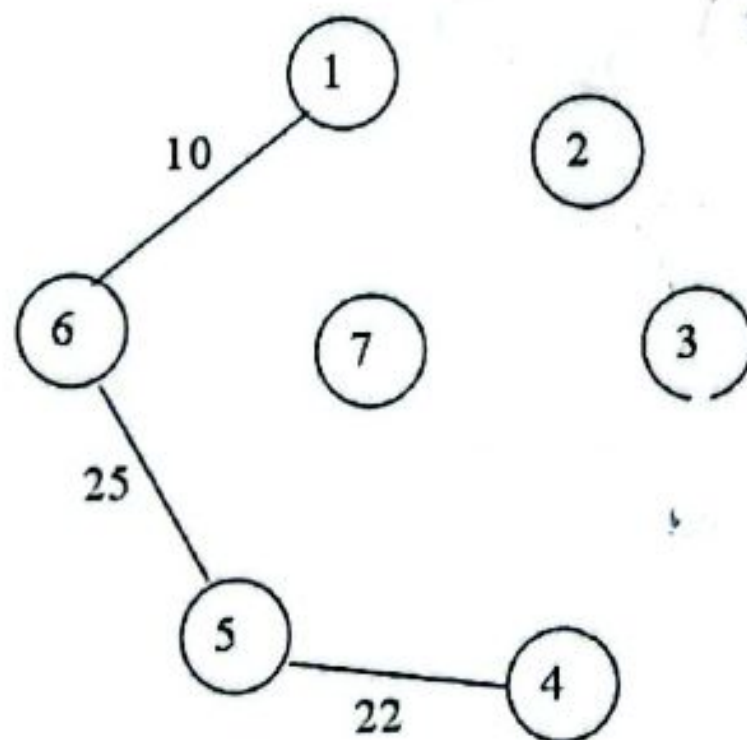
Step 1:



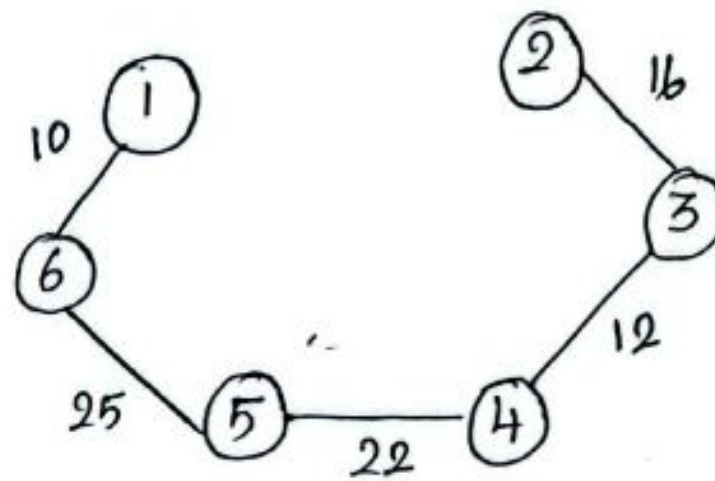
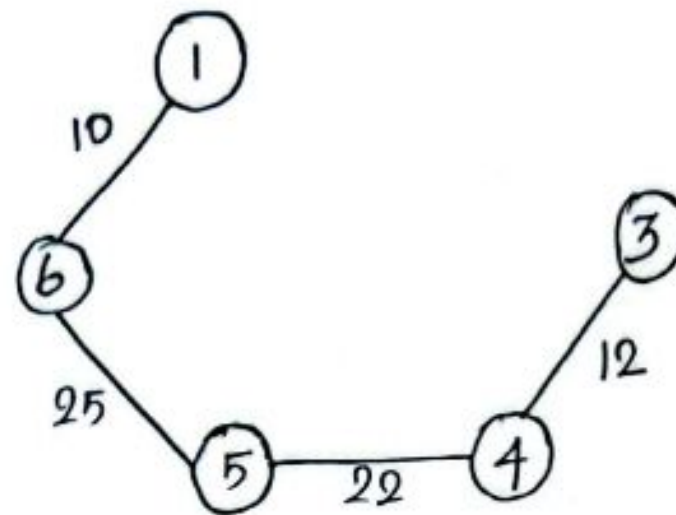
step:2



Step :3

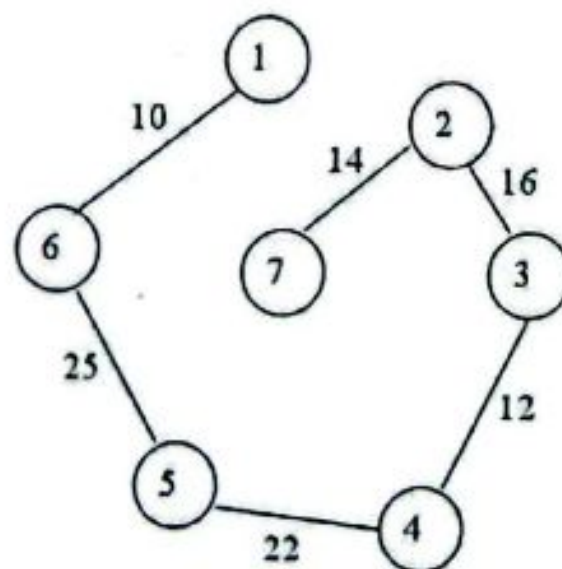


Step : 4



Step 5:

Step 6:



9. Briefly explain the Dijkstra's shortest path algorithm (Nov 2019).

Procedure Dijkstra

{ Dijkstra computes the cost of the shortest path from vertex 1 to every vertex of a directed graph } begin

1. $s = \{1\};$

2. for $i = 2$ to n do

3. $D[i] := c[1, i];$ {initialize D}

4. for $i = 1$ to $n-1$ do begin

▪ choose a vertex w in $V-S$ such that $D[w]$ is minimum

5. Add w to S

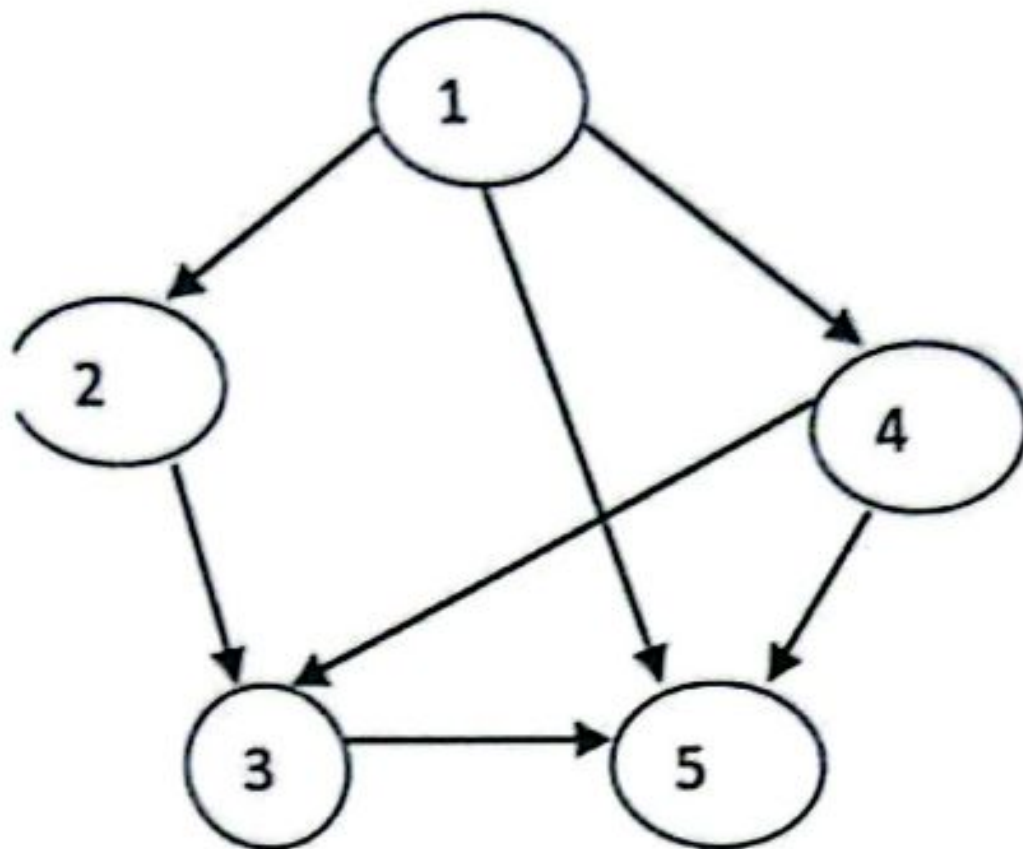
6. for each vertex v in $V-S$ do

7. $D[v] := \min(D[v], D[w] + c[w, v]);$

8. end

end{Dijkstra}

Example 2:



Cost adjacency matrix:

1 2 3 4 5

1 9999 10 9999 30 100

2 9999 9999 50 9999 9999

3 9999 9999 9999 9999 10

4 9999 9999 20 9999 60

5 9999 9999 9999
9999 9999 S={1}

V={1,2,
3,4,5}

For i=2
to 5 do

i=2

D[2]=1

0 i=3

D[3]=9

999

i=4

D[4]=3

0 i=5

D[5]=1

00 for

i=1 to

4 do

D[3]=60

D[5]=30

i=1 V-

S={2,3,4

,5}

D[w]=1

0;

w=2(mi

nimum)

S={1,2}

For each vertex v in V-S i.e)V-

S={3,4,5}=>for v=3,4,5 When v=3

D[3]=min(D[3],D[2]+c[2,3])

D[3]=min(9999,10+50)

D[3]=60

1

2

4

3 5

When $v=4$
 $D[4] = \min(D[4], D[2] + c[2,4])$
 $D[4] = \min(30, 10 + 9999)$
 $D[4$
 $] = 3$

0

Wh

en

 $v=5$
 $D[5] = \min(D[5], D[2] + c[2,5])$
 $D[5] = \min(100, 10 + 9999)$
 $D[5] = 100$
 $D[2] = 10 \ D[3] = 60 \ D[4] = 30$
 $D[5] = 100 \ i = 2$
 $V-S = \{3, 4, 5\}$
 $D[w] = 30;$
 $w = 4$ (minimum)

 $S = \{1, 2, 4\}$

For each vertex v in $V-S(3,5) \Rightarrow$ for

 $v=3, 5$ When $v=3$
 $D[3] = \min(D[3], D[4] + c[4,3])$
 $D[3] = \min(60, 30 + 20)$
 $D[3$
 $] = 5$

0

Wh

en

 $v=5$

$$D[5] = \min(D[5], D[4] + c[4, 5])$$

$$D[5] = \min(100, 30 + 60)$$

$$D[5] = 90$$

$$D[3] = 50$$

$$D[5] = 90 \quad i = 3$$

$$V-S = \{3, 5\}$$

$$D[w] = 50;$$

$$w = 3(\text{minimum})$$

$$S = \{1, 2, 4, 3\}$$

For each vertex v in V -

$S = \{5\} \Rightarrow$ for $v = 5$ When $v = 5$

$$D[5] = \min(D[5], D[3] + c[3, 5])$$

$$D[5] = \min(90, 50 + 10)$$

$$D[5] = 60$$

$$D[5] = 60$$

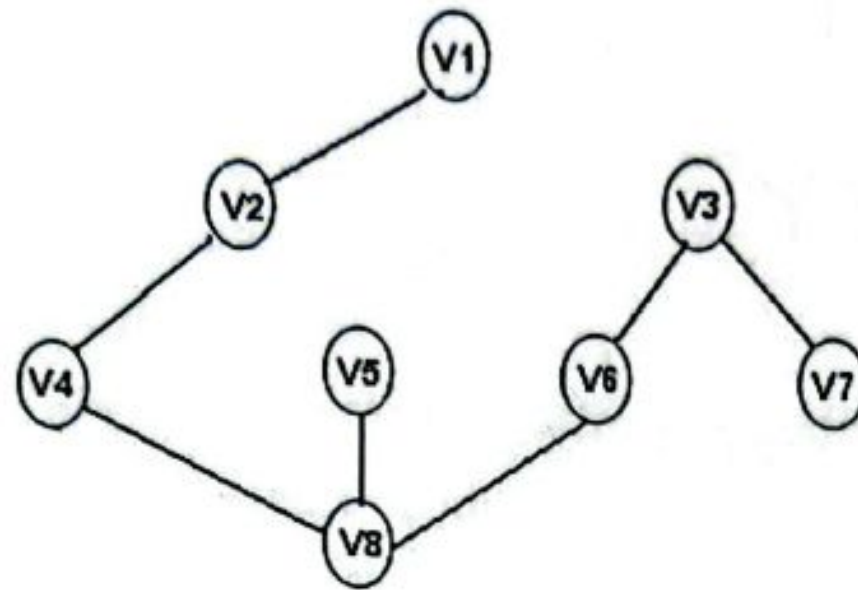
$$i = 4$$

V -

$S = \{$

$5\}$

The below diagram shows the Depth first spanning tree

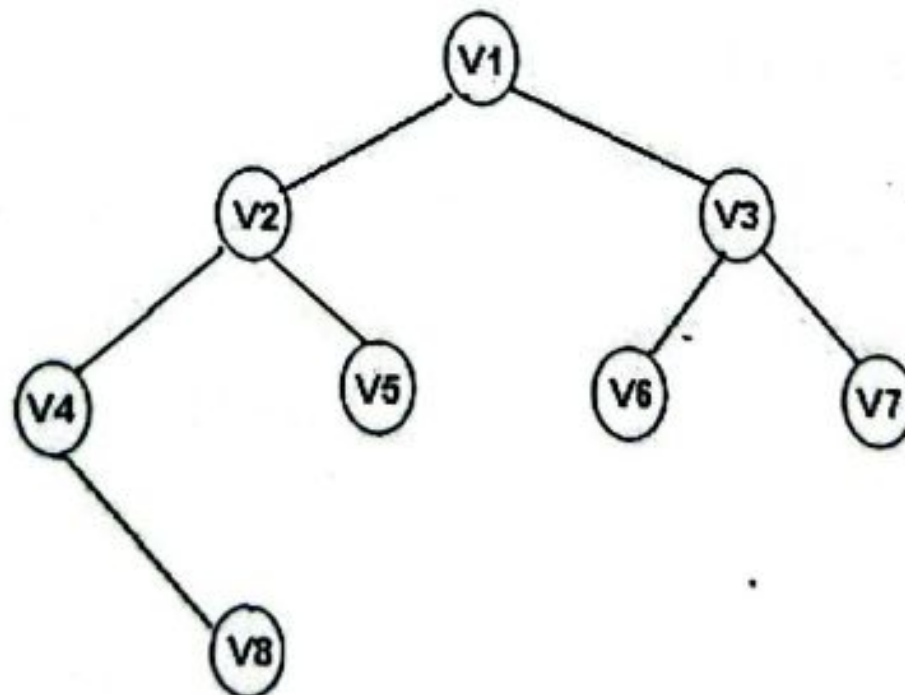


■ BREADTH FIRST SPANNING TREE :

When BFS are used the edges of T form a spanning tree.

The spanning tree resulting from a call to BFS is known as a breadth first spanning tree

The below diagram shows the breadth first spanning tree



SPANNING TREE

A tree is a spanning tree of a connected graph $G(V, E)$ such that,

Every vertex of G belongs to an edge in T and

The edges in T form a tree.

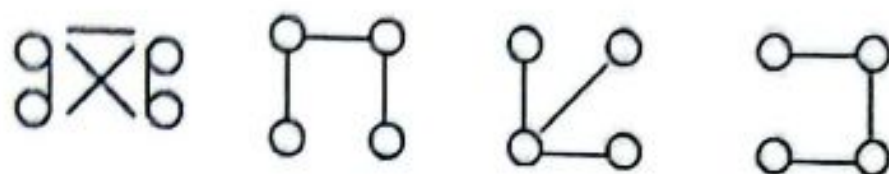
CONSTRUCTING A SPANNING TREE

▣ Let us see how we construct a spanning tree for a given graph.

Take any vertex v as an initial partial tree and edges one by one so that each edge gives a new vertex to the partial tree.

In general if there is a vertex in the graph we shall construct a spanning tree in $(n-1)$ steps i.e. $(n-1)$ edges are needed to be added.

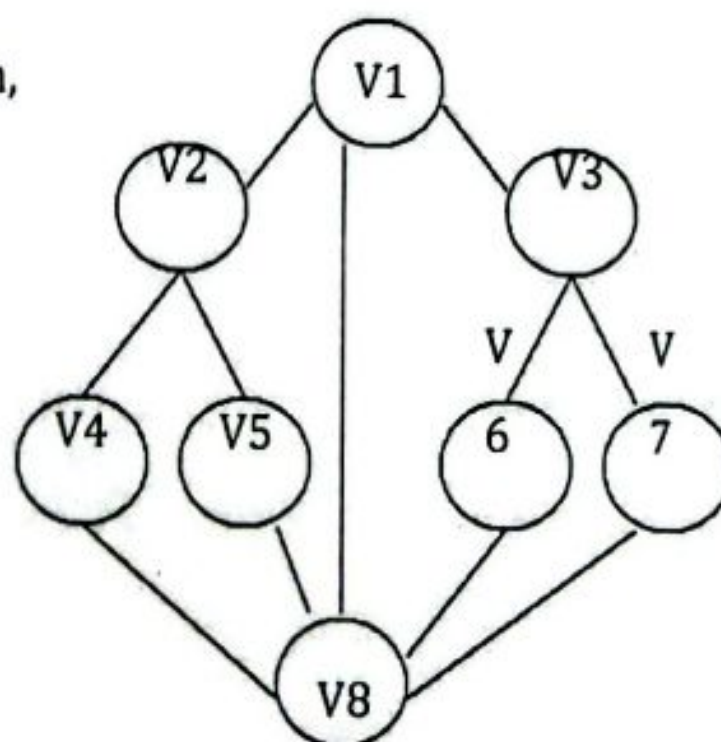
The following figure shows a complete graph and three of its spanning trees.



When either DFS or BFS are used the edges of T form a spanning tree.

The spanning tree resulting from a call to DFS is known as a **depth first spanning tree**. When BFS is used the resulting spanning tree is called a **breadth first spanning tree**.

Consider the graph,



The following figure shows the spanning trees resulting from depth first add breadth first search starting at vertex V1 in the above graph.

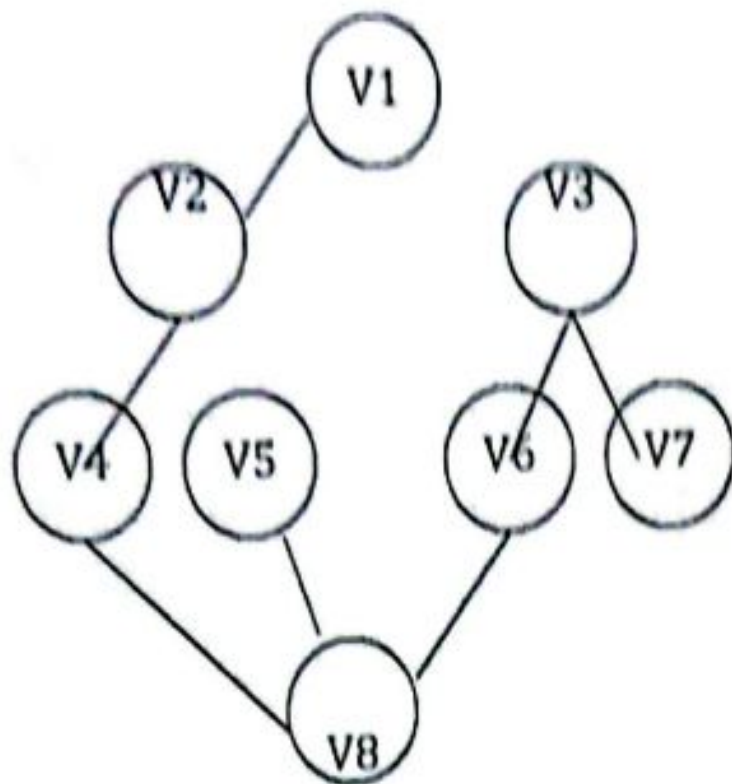


Fig.: DFS (I) spanning tree.

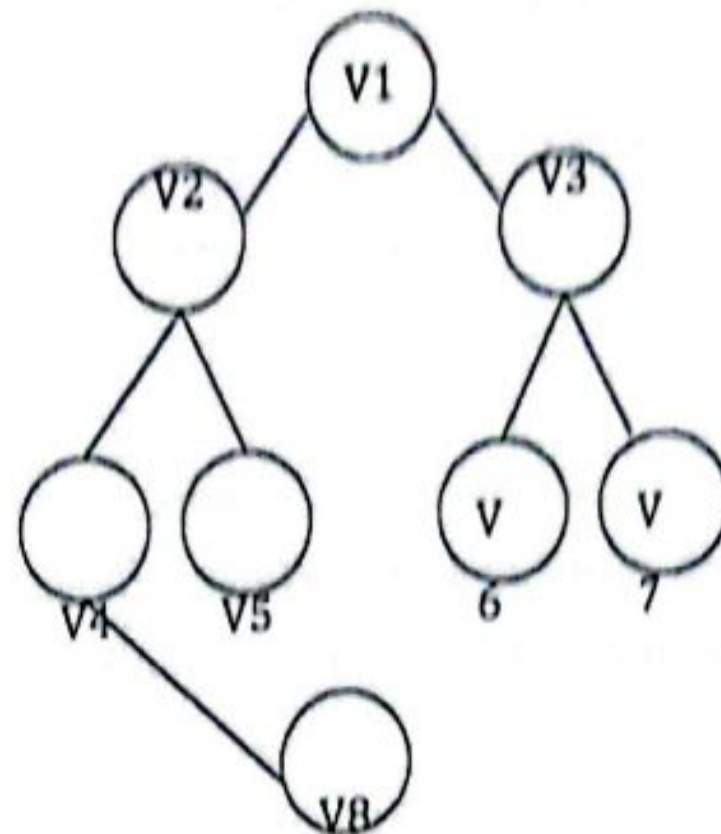


Fig.: BFS (I) Spanning tree

MINIMUM COST SPANNING TREES (APPLICATIONS OF SPANNING TREE):

BUILDING A MINIMUM SPANNING TREE

- If the nodes of G represent cities and the edges represent possible communication links connecting two cities then the minimum number of links needed to connect the n cities is $n-1$. The spanning tree of G will represent all feasible choices.

- ▣ This application of spanning tree arises from the property that a spanning tree is a minimal sub-graph

G' of G such that $V(G') = V(G)$ and G' is connected where minimal sub-graph is one with fewest number of edges.

- ▣ The edges will have weights associated to there i.e. cost of communication. Given the weighted graph, one has to construct a communication links that would connect all the cities with minimum total cost.
- ▣ Since, the links selected will form a tree. We are going to find the spanning tree of a given graph with minimum cost. The cost of a spanning tree is the sum of the costs of the edges in the tree.